后训练学习计划 · 第一周（重制版 v2）

> 本周定位：**Phase 0 启动周——把地基一次打牢**。主题是 GPU 环境 + micrograd 亲手复刻 + 手写 attention + 知识自测。
> 本版按你新的节奏重排：每天投入 **≥6 小时**，一天吸收原计划约 1.5 天的内容，全周 7 天（6 天学习 + 1 天缓冲验收），总预算约 41.5 小时、花费 ≤¥20。
> 用法说明：每天先看「今日总览」，再从「任务 1」开始逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗红字 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	GPU 环境全通 + micrograd 视频(上)	GPU 冒烟跑通（wandb 有记录或离线包）	6h
Day 2 周二	micrograd 视频(下) + 完整跟写	engine/nn/train_moons 三件套跑通、acc≥90%	6.5h
Day 3 周三	盲写复刻 + 数值梯度对拍	mine.py 收敛 + GitHub 链接	6h
Day 4 周四	限时自测 6 题 + 错题映射 + 博客成文	week1-gaps.md + 博客草稿	6h
Day 5 周五	手写 attention(单头→多头→可视化)	attention.py 单测全绿 + 博客发布	6h
Day 6 周六	PyTorch 桥接 + 对拍 + 周复盘	torch 对照版代码 + review.md	7h
Day 7 周日	缓冲补漏 + 总验收 + 下周预习	验收 ≥6/7 项通过	4h

> 注：GPU 只在 **Day 1** 需要（micrograd 是纯 CPU 项目，Day 2 起全在本地）。4090 按量约 ¥1.5-2.5/小时，Day 1 实际用卡 2-3 小时 ≈ ¥5-8，全周 **≤¥20**（含充值手续费）。Day 5 的 attention、Day 6 的 torch 对拍都在本地 CPU 完成。
> 注：每天结束前花 10 分钟把卡点写进 `NOTES.md`（见 Day 1 任务 5），这些是 Day 4 博客的原材料，别欠账。

> 注：本周最大风险不是难度而是**时间**：Day 2 和 Day 3 各被低估约 1-2 小时。砍单顺序（实在做不完时按这个砍）：Day 2 任务 3 画图 → Day 5 任务 4 热力图 → Day 3 任务 3 推 GitHub（延到 Day 6）→ **其余不许砍**。
> 注：本计划**故意不含** SFT/LoRA/RLHF/DPO 等真正的后训练环节，第一周只做 autograd 与 attention 两块地基。别提前跳到调包微调——那正是这周要避开的弯路。

二、开始前：本地环境、目录与工具（约 25 分钟，放在 Day 1 之前一晚做完）

0. **本地 Python 环境**（Day 2 起所有代码都在本地跑，原计划漏了这步，必须提前做）。装 Python 3.10+（有 Anaconda 就用 conda），然后：

```
cd E:/study/posttrain
python -m venv .venv
# PowerShell:  .venv\Scripts\Activate.ps1      Git Bash:  source .venv/Scripts/activate
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
pip install numpy matplotlib scikit-learn torch
python -c "import numpy,sklearn,matplotlib,torch;print(torch.__version__)"
```

> 注：torch 装 CPU 版即可（国内直接 pip 通常给的是 CUDA 版，也能跑）。`pip install torch --index-url https://download.pytorch.org/whl/cpu` 是更小的下载量，二选一，别在这上面耗时间。
> 注：**别装在 conda 的 base 环境里**——conda 的 MKL 与 torch 自带的 OpenMP 会打架（`OMP Error #15`，Day 6 会遇到），上面这个 venv 就是为了躲开它。已经装了 anaconda 也照样能用，只是这个练习独立建一个 venv 更省心。

1. 建目录（注意：bash 里反斜杠会被吞，用正斜杠；cmd 里用反斜杠）：

```
mkdir -p E:/study/posttrain/{day1,day2,day3,day4,day5,day6}
```

2. 装 git 并配置身份（Day 3 要推 GitHub，临时配会卡住）：`git config --global user.name` / `user.email`，并登录一次 GitHub 的凭据（HTTPS 推送需要 Personal Access Key，或装 GitHub CLI 用 `gh auth login`）。
3. 本周你会用到的账号（Day 1 任务 1 注册）：AutoDL（租 GPU）、wandb（训练记录）、GitHub、Hugging Face、魔搭 ModelScope。建议用同一个邮箱+密码管理器记录。

Day 1 • 周一：GPU 环境全通 + micrograd 视频（上）

今日总览（6h）：今天只有一个目标——**晚上 10 点前，你的 wandb 账号里出现一条从 GPU 实例发出的训练记录**。上午注册账号 + 租卡 + 环境冒烟（约 2h），下午开始看 Karpathy 的 micrograd 视频上半场（约 3.5h），最后 10 分钟写收工笔记。 micrograd 是后训练一切知识的起点：反向传播不是魔法，是一个链式法则记账系统，本周你要亲手复刻它。

任务 1：注册账号（约 30 分钟）

1. 打开 `https://www.autodl.com` → 注册 → 实名认证（支付宝扫码即可）→ 先充 20 元。
2. 打开 `https://wandb.ai` → Sign up（可直接用 Google 账号登录）→ 登录后进入 `https://wandb.ai/authorize`，**复制页面上显示的 API Key**（一串 32 位字符），先粘到记事本里。
3. 打开 `https://github.com` → 注册（Day 3 要交作业，别拖到那天）。
4. 打开 `https://huggingface.co` → 注册（邮箱验证）。 国内如打不开，跳过，本周不强制。
5. 打开 `https://modelscope.cn` → 注册（淘宝/支付宝账号可直接登录）。

完成判据：AutoDL/wandb/GitHub/魔搭 都能正常登录（HF 打不开不算失败）；wandb API Key 已保存到记事本。

> 注：Hugging Face 打不开不影响本周任务，不要在这上面耗超过 10 分钟。魔搭是它的国内平替。

任务 2：租一台 4090 并连上终端（约 40 分钟）

1. AutoDL 控制台 → 算力市场 → 地区选「西北 B 区」或任意有货的 → 卡型筛选 **RTX 4090（24G）** → 计费选**按量计费**。
2. 镜像选「基础镜像」→ PyTorch → 选 **2.1 以上 + Python 3.10 + CUDA 12.1** 的那个（名字里带 pytorch2.x/cuda12.1 即可）。
3. 创建实例 → 等状态变「运行中」→ 点「JupyterLab」进入，打开 Terminal。
4. 在 Terminal 里确认 GPU 可用：

```
nvidia-smi
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

预期输出第二行类似 `2.1.0+cu121 True`。`True` **是硬性要求**。

> 注：「跑完就关机」规则从今天开始生效：每次离开终端前，在 AutoDL 控制台点**关机**（不是销毁！）。关机后只收少量磁盘费，重新开机数据还在。**销毁会连数据盘一起清空**，任何时候不要点销毁。
>
> 注：4090 经常没货——**不要刷页面等卡**。任选一张能开机的卡（3090/4090D/A100 都行），今天的任务只是矩阵乘冒烟，不挑卡。
>
> 注：实例上的代码随时可能没了（误操作/平台回收）。今天在实例上写的任何东西，收工前用 JupyterLab 的「下载」按钮存回本地 `day1/`，或直接在终端里 `git push`。**本地必须有一份**。

任务 3：环境冒烟 wandb 上报（约 40 分钟）

1. 在 JupyterLab 里新建文件 `/root/env_check.py`，内容原样复制：

```
# env_check.py — 环境冒烟:GPU、矩阵乘、wandb 上报三件事
import torch, time, wandb

assert torch.cuda.is_available(), "CUDA 不可用,检查镜像是否选对"
print("torch:", torch.__version__, "| device:", torch.cuda.get_device_name(0))

run = wandb.init(project="week1-env", name="env-check", mode="online")
x = torch.randn(4096, 4096, device="cuda")
for i in range(10):
    t0 = time.time()
    y = x @ x
    torch.cuda.synchronize()
    wandb.log({"matmul_sec": time.time() - t0, "step": i})
    print(f"step {i}: {time.time()-t0:.3f}s")
run.finish()
print("ALL OK")
```

2. 终端里登录 wandb（粘贴你任务 1 存的 API Key）：

```
pip install wandb -q
wandb login <你的API Key>
```

3. 运行：

```
python env_check.py
```

完成判据：终端打印 `ALL OK`；wandb 首页 → week1-env 项目 → 能看到 10 个 matmul_sec 数据点（或已按下面注释切成 offline 模式并记进 gaps）。

> 注：**wandb 在国内实例上连不上是常态，不要把这当成你的错**。按顺序试：① `pip install wandb -U` 重登；② `export WANDB_MODE=offline`，跑通后 `wandb sync <离线目录>` 补传（sync 要在有代理/网络好的时候做，可以回家再补）；③ 试 AutoDL 的学术加速 `source /etc/network_turbo`（只对 github/hf/pypi 等生效，对 wandb 常无效）。**今天最多耗 30 分钟**：拿不到云端记录就改用 `WANDB_MODE=offline`，只要终端打印 `ALL OK` 且本地生成了离线目录，任务 3 就算过——把「周末补 sync」写进 gaps.md。

任务 4：micrograd 视频上半场（约 3.5 小时）

1. 打开视频：Karpathy 的 **「The spelled-out intro to neural networks and backpropagation: building micrograd」**（YouTube 原链 `https://www.youtube.com/watch?v=VMj-3S1tku0`；B 站搜「micrograd karpathy」有中文字幕搬运，非官方但够用）。**全片约 1 小时 58 分**——别按原来说的 2 小时 25 分排时间。
2. 今天只看上半场。结束点**按内容切、不要掐时间戳**：看到他把 `Value.backward()` 的拓扑排序写完、`+=` 那个 bug 也讲完，正要开始写 `tanh` / `Neuron` 时停（大致在 1:15-1:25 处）。剩下的全部留给明天的任务 1。
3. 看法要求（**这是本任务的硬性要求**）：
   - 全程用电脑跟看，**他写一行你写一行**，文件放 `E:\study\posttrain\day1\follow.py`。
   - 每个概念出现时暂停 30 秒，用自己的话在 `E:\study\posttrain\NOTES.md` 里写一行（中英文都行）。
   - 视频里用 `graphviz` 画计算图那段（约 42 分钟处）：跟着装 `pip install graphviz` 并把他的 `draw_dot` 函数抄进 follow.py；本机渲染报错（缺 Graphviz 系统包）就**跳过画图**，不要装系统包浪费时间。
4. 收工前在本地跑一遍 follow.py，确认没有报错。

完成判据：follow.py 里有一个 `Value` 类，`+ *` 能前向、`backward()` 能跑通一个手工小计算图；NOTES.md 里有 ≥8 条自己的话。

> 注：今晚不需要看懂 100%。`+=` 与 `=` 的区别、拓扑排序为什么必须反着遍历，这两个问题如果卡住，写进 NOTES.md 带着睡——Day 2 的跟写和 Day 3 的盲写会替你把它想通。

任务 5：收工笔记（约 10 分钟）

在 `E:\study\posttrain\NOTES.md` 里写三行：今天最容易的一个卡点、一个明天要回答的问题、明晚计划开始时间。

今日验收：wandb 有记录（云端或 offline 待补）+ follow.py 存在 + NOTES.md ≥8 条。三项齐了才收工；到 23:30 还没齐，**把缺的写进 gaps.md 再睡**（连续熬夜会把后面四天的正确率拖垮，比缺一项更亏）。

Day 2 • 周二：micrograd 视频（下）+ 完整跟写

今日总览（6.5h）：看完视频剩下的部分（tanh、整个 MLP、训练循环），然后把 micrograd 的完整实现**跟着视频逐行敲一遍**，晚上让它在一个二维数据集上训练收敛。今天结束时你会拥有一个「自己敲出来的、能训练的神经网络」——虽然它只有 40 行。

> 注：昨天的 GPU 实例今天用不上了（micrograd 是纯 CPU 项目），**记得把实例关机**（顺手确认控制台显示「已关机」，别按小时继续扣）。今天全部在本地写。视频比原估的短约 40 分钟，省出来的时间优先给任务 2 的默写，不要拿去开新坑。

任务 1：视频下半场（约 1.5h，其中视频只剩约 45 分钟）

1. 从昨天停的地方（`tanh` 之前）看到结尾，全片约 1:58:00 结束。今天覆盖：`Value.tanh`、`Neuron/Layer/MLP` 三个类、moons 数据集、训练循环。
2. 继续「他写一行你写一行」，追加到 `day1/follow.py`。
3. 视频结尾他把 loss 压到很低、训练集 acc 大约 90%+ ——你跟到类似效果再停。

完成判据：follow.py 运行后 loss 逐步下降到 0.1 以下；NOTES.md 新增 ≥5 条（重点记「拓扑排序为什么必要」）。

任务 2：脱离视频整理成自己的三个文件（约 2.5h）

合掉视频，只对照 follow.py 和 micrograd 原仓库（`https://github.com/karpathy/micrograd`），把代码整理成自己的干净版本，放 `day2/` 下三个文件。

文件 1： `day2/engine.py`（Value 引擎，核心约 85 行，其中 20 行是机械的反射式算子）——先自己默写框架，卡住才看参考：

```
import math

class Value:
    """标量自动微分引擎:存 value、父节点、局部梯度函数"""
    def __init__(self, data, _children=(), _op=""):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), "+")
        def _backward():
            self.grad += out.grad          # 加法:梯度原路分发
            other.grad += out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), "*")
        def _backward():
            self.grad += other.data * out.grad   # 乘法:交叉相乘
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def tanh(self):
        t = math.tanh(self.data)                # 别用 (exp(2x)-1)/(exp(2x)+1),x 大时会溢出
        out = Value(t, (self,), "tanh")
        def _backward():
            self.grad += (1 - t**2) * out.grad   # tanh 导数 1-t^2
        out._backward = _backward
        return out

    # ---- 以下 7 个都是上面几行的机械变形。默认容易漏,漏了 train_moons.py / gradcheck.py 直接报错 ----
    __radd__ = __add__                          # 让 sum(list_of_Value) 能用(它会先加整数 0)
    __rmul__ = __mul__                          # 让 2*Value、-yi*pred 这类"数字在左边"的写法能用
    def __neg__(self):
        return self * Value(-1.0)
    def __sub__(self, other):
        return self + (-other if isinstance(other, Value) else Value(-other))
    def __rsub__(self, other):
        return Value(other) + (-self)
    def __truediv__(self, other):
        return self * (other if isinstance(other, Value) else Value(other)) ** -1
    def __pow__(self, n):
        out = Value(self.data ** n, (self,), f"**{n}")
        def _backward():
            self.grad += (n * self.data ** (n - 1)) * out.grad
        out._backward = _backward
        return out

    def backward(self):
```

```
        # 拓扑排序:保证先算子节点再算父节点的梯度
        topo, visited = [], set()
        def build(v):
            if v not in visited:
                visited.add(v)
                for child in v._prev:
                    build(child)
                topo.append(v)
        build(self)
        self.grad = 1.0
        for v in reversed(topo):
            v._backward()

    def __repr__(self):
        return f"Value({self.data})"
```

> 注：`__add__` 里 `self.grad +=` 的 `+=` 而不是 `=`，是因为一个节点可能被多个表达式共用（视频里的 bug 案例）。默写时这里错最多，注意。

文件 2： `day2/nn.py`（Neuron/Layer/MLP 三个类，约 40 行，照视频结构默写）。四个容易漏的点：`Neuron.__call__(x)`（否则 `model([x1,x2])` 调不通）、`MLP.parameters()` 要把三层参数一层层 yield 出来、`Layer` 只有一个神经元时返回的是 Value 本身（**所以 train_moons.py 里不要再写 `[0]`**）、**最后一层不要再过 tanh**（视频里的 `MLP.forward` 只在层**之前**过非线性）。若输出层也过 tanh，预测被压进 [-1,1]，平方 hinge 的间隔推不满，acc 会低一档。

文件 3： `day2/train_moons.py` ——用 sklearn 造 moons 数据训练：

```
import random
random.seed(42)
from sklearn.datasets import make_moons
from engine import Value
from nn import MLP

X, y = make_moons(n_samples=100, noise=0.15, random_state=42)   # random_state 必须写,否则 Day 6 对不上
y = [2*int(yy) - 1 for yy in y]             # 标签转 -1/+1
model = MLP(2, [16, 16, 1])                 # 2入 16 16 1出

def forward_all():
    return [model([x1, x2]) for x1, x2 in X]          # 最后一层只有 1 个神经元,返回的就是 Value 本身

def loss_fn(preds):
    losses = [(1 - yi * pred) ** 2 for yi, pred in zip(y, preds)]   # 平方 hinge:要求 yi*pred→1
    data_loss = sum(losses) / len(losses)
    reg = 1e-4 * sum(p * p for p in model.parameters())
    return data_loss + reg

for epoch in range(300):
    preds = forward_all()
    loss = loss_fn(preds)
    for p in model.parameters():
        p.grad = 0.0
    loss.backward()
    lr = 0.1 * (1 - 0.9 * epoch / 300)          # 学习率衰减:0.1 → 0.01
    for p in model.parameters():
        p.data -= lr * p.grad
    if epoch % 50 == 0 or epoch == 299:
        acc = sum((pred.data > 0) == (yy > 0) for pred, yy in zip(forward_all(), y)) / len(y)
        print(f"epoch {epoch} loss {loss.data:.4f} acc {acc:.2%}")
```

> 注：**loss 里必须出现标签 `yi`**。写成 `(1 - pred)**2` 会让所有点都被推向 +1，模型把两类全预测成正类，acc 永远停在 50%——这一版草稿就犯了这个错。
>
> 注：acc 那行也别写成 `(pred.data > 0) == yy`。Python 里 bool 是 int 的子类，`True == 1` 成立但 `True == -1`、`False == -1` 全为 False，于是负类永远算错，**acc 上限被钉死在 50%**，你会拿着一份正确收敛的代码去 debug 一晚上。判正误要统一比较 `> 0` 之后的布尔值。
>
> 注：**学习率不要照抄视频里的 1.0**。平方 hinge 的梯度随 logit 线性增长，实测 lr=1.0 约在 127 轮就把 loss 推到 1e150 并溢出崩溃；上面这份 0.1→0.01 衰减、300 轮的写法实测约 150 轮就到 acc 97%（`epoch 50 loss 0.2195 acc 92%` → `epoch 299 loss 0.1480 acc 97%`），判据里的 95% 是够得着的。想复现视频里那种「lr=1.0 也不炸」，得用他另一份带 tanh 压扁的 loss：`(1 - yi*pred).tanh() ** 2`——tanh 把每个样本的损失限在 [0,1]，越过间隔后梯度自动趋零，所以敢用大 lr（实测 200 轮：lr=1.0 → 91%，lr=0.3 → 94%）。**两种都跑一遍并记下各自最终 acc**：损失函数的形状直接决定优化的稳定性，这个体感比看十遍讲解有用。
>
> 注：micrograd 是纯 Python 逐个标量算的，**100 个点 × 337 个参数 × 300 轮 ≈ 4-5 分钟**，屏幕很久不动属正常，别以为是死循环去 Ctrl+C。想快就把打印间隔调大、或先跑 100 轮验证趋势。

完成判据：`python train_moons.py` 打印的最终 **acc ≥ 95%（训练集）**。**掉到 90-95% 之间也允许收工**，把「acc 不够」记进 gaps，盲写的 Day 3 再按 95% 要求——今天真正的产出是「跑通」，不是刷分。

> 注：acc 卡在 50% 不动 20 分钟的话，按这个顺序查：① loss 里有没有标签 `yi`（上面第一条注）② acc 比较有没有踩 bool/-1 的坑（第二条注）③ 梯度有没有清零 ④ `+=` 写成了 `=` ⑤ backward 拓扑顺序。限时 40 分钟，解决不了就把两个文件与 follow.py 逐行 diff。

任务 3：加可视化（约 30 分钟，超时 40 分钟直接砍到 Day 7）

新建 `day2/plot.py`：训练完后用 matplotlib 把 100 个点和模型决策边界画出来（把平面 400 个网格点过一遍模型，`plt.contourf`）。存图 `day2/moons.png`。**这是本周可选装饰项**，砍掉不影响主线，但博客要用它，优先在 Day 2 做。

完成判据：moons.png 能明显看到两条月牙被分界线分开。

任务 4：收工笔记（10 分钟）

NOTES.md 回答两个问题：①「梯度为什么要 += 而不是 =」②「backward 里为什么先 build 拓扑再 reversed」。

今日验收：moons.png（或已砍到 Day 7）+ train_moons 跑通且 acc 达标 + NOTES ≥5 条新条目。

Day 3 • 周三：盲写复刻 + 数值梯度对拍

今日总览（6h）：全周最重要的三天之一。上午**关掉所有参考**，从空文件开始默写整个 micrograd（约 3h，卡住就记 NOTES 硬扛）；下午与原版 diff 修错、推 GitHub，再做数值梯度对拍单测。今天结束时你拥有一个**可以写进简历的 GitHub 仓库**。

> 注：盲写的目的不是背代码，是逼你暴露「以为懂了但没懂」的地方。**今天写得痛苦是正常的、是设计如此。**

任务 1：盲写 mine.py（约 3h，含卡点记录）

1. 合掉所有窗口，只开一个空文件 `day3/mine.py` 和 NOTES.md。
2. 从零默写：Value 类（`__add__` / `__mul__` / `__radd__` / `__rmul__` / `__neg__` / `__sub__` / `__pow__` / `__truediv__` + tanh + `backward()` 拓扑排序）+ Neuron/Layer/MLP + 一个 30 轮训练循环（数据用 `random` 造两个高斯团即可，不用 sklearn）。
3. 规则：
   - **不许打开任何参考文件/网页**（micrograd 仓库、follow.py、engine.py 全部关掉）。
   - 卡住超过 5 分钟 → 把「卡在哪、为什么」写进 NOTES.md 的 `## 盲写卡点` 小节，然后继续硬想；连续卡 15 分钟才允许瞄一眼自己的 day2（不是原仓库）。
4. 训练循环跑通、loss 下降即为盲写完成（不要求 acc）。

> 注：`__pow__/__sub__/__truediv__` 这三个**不许省**——train 循环里的 `/ len(...)` 和 gradcheck 里的 `x - h` 都靠它们，省了就是当天多花 20 分钟 debug。真要省，就把 loss 改成只用 `+ *` 的写法（自己想想怎么写）。

完成判据：mine.py 独立运行且 loss 下降；NOTES.md 盲写卡点 ≥3 条（**这些就是博客素材**）。

任务 2：diff 与修复（约 1h）

1. 打开原仓库 `https://github.com/karpathy/micrograd/blob/master/micrograd/engine.py` 与你的 mine.py 对比。
2. 在 NOTES.md 写「三个差异」：命名差异、实现差异、我多写/少写了什么。
3. 修复所有语义 bug，重跑确认 loss 收敛。

任务 3：推 GitHub（约 40 分钟）

1. 新建仓库 `micrograd-from-scratch`（Public），`day3/` 内容为根目录：`mine.py`、`train.py`、`gradcheck.py`（任务 4）、`README.md`。
2. README 写四段：这是什么 / 与原版的差异 / 怎么跑 / 学到什么（从 NOTES 盲写卡点里挑 3 条）。
3. `git init && git add -A && git commit -m "micrograd from scratch" && git push`（仓库建好后页面上有现成命令）。

> 注：push 卡在认证上是这一天的头号时间杀手（HTTPS 要 PAT 不是密码；`gh auth login` 浏览器方式最省事）。**Day 3 一开始就先 `git push` 一个空 README 打通认证**，别等到最后一小时。若认证 40 分钟没通，先 `git bundle` /zip 备份本地，改到 Day 6 补推，不要牺牲盲写当天的复盘质量。

完成判据：GitHub 仓库链接能打开，README 渲染正常。**把链接记到 NOTES.md 顶部。**

任务 4：数值梯度对拍 gradcheck.py（约 1h）

数值梯度是验证一切自动微分的金标准。新建 `day3/gradcheck.py`：

```
"""数值梯度 vs 反向传播 对拍:两者误差应 < 1e-5"""
import random
from mine import Value

random.seed(0)

def f(v):                                    # v 是 Value
    return (v * 3 + Value(2)) * v + v.tanh()   # 随便一个复合函数

def numerical_grad(f, x_data, h=1e-6):       # 中心差分:每次都用全新的 Value,避免梯度串味
    return (f(Value(x_data + h)).data - f(Value(x_data - h)).data) / (2 * h)

x = Value(0.7)
f(x).backward()
analytic = x.grad
numeric = numerical_grad(f, x.data)
diff = abs(analytic - numeric)
print(f"analytic={analytic:.8f} numeric={numeric:.8f} diff={diff:.2e}")
assert diff < 1e-5, f"梯度对拍失败 diff={diff}"
print("GRADCHECK OK")
```

完成判据：`python gradcheck.py` 打印 `GRADCHECK OK`。把这个文件也 push 上 GitHub。

> 注：h=1e-6 的中心差分，diff 一般在 1e-9~1e-7 量级。如果大得离谱（>1e-3），按 `__pow__` → `__truediv__` → tanh 反向 → 拓扑顺序 的顺序逐个隔离：把 f 换成只用一个算子的最简式，看哪个算子对不上。
> 注（选做，博客里很加分）：把 gradcheck 升级成**对整个 MLP 的所有参数抽样对拍**——随机初始化网络，`loss_fn()` 求 backward 后，随机挑 5 个参数 p，把 p.data 加减 h 各跑一次前向算数值梯度。这一步做完，你对「autograd 到底在算什么」就没有模糊地带了。

任务 5：收工笔记（10 分钟）

NOTES.md 写：盲写时最痛的 1 个点 + 为什么数值梯度能当裁判（一句话）。

今日验收：mine.py 收敛 + GitHub 链接 + gradcheck OK + 盲写卡点 ≥3 条。

Day 4 • 周四：限时自测 + 错题映射 + 博客成文

今日总览（6h）：今天不动代码主线。上午闭卷自测 6 道基础题（90 分钟），暴露「视频看懂了但手推写不出」的空洞；下午把错题映射到具体补课日期，然后把前三天的素材（盲写卡点、对拍、决策边界图）写成知乎文章草稿。**输出物是后训练求职的门面，不能省**。

任务 1：闭卷自测 90 分钟（写在本子上或 md 里，不查任何资料）

1. 手推反向传播：网络 `y = tanh(w2 · tanh(w1 · x + b1) + b2)`，写出 loss 对 w1 的梯度表达式（链式法则逐步展开，标注每步维度）。
2. 写出 softmax 的定义；证明 softmax + 交叉熵的梯度是 `p - y`；说明为什么实现时要把它们合并成一个算子。
3. 写出 KL 散度定义 D_KL(P||Q)；说明它与交叉熵、熵的关系；为什么训练时用 CE 而不是直接最小化 KL。
4. 画出 Transformer decoder block 的结构图（至少标出：LayerNorm、多头注意力、FFN、残差连接的位置），并写出每个子层的输入输出维度关系。
5. 写出 Adam 的两个动量更新式与最终参数更新式；解释为什么要做偏差修正。
6. 用伪代码（NumPy 风格）写单头 self-attention 的前向（Q、K、V → 缩放点积 → softmax → 加权求和），并指出 causal mask 应该加在哪一行。

完成判据：6 题全部写了答案（写错没关系，**空着才算失败**）。计时 90 分钟，到点停笔。

> 注：**这是一次基线诊断，不是考试，不计分、不代表失败**。第 2/3/4/5 题（softmax+CE、KL、Transformer 结构、Adam）本周压根没学过，答不出完全在预期之内；它们出现在这里只是为了让你在开工前就看到「后训练要用的地基长什么样」，并把漏洞量化。**只有第 1、6 题是本周内必须能补上的**。别为不会的题加练到凌晨——那是计划要背的债，不是你今天的债。

任务 2：对答案 + 错题映射（约 1.5h）

步骤 1：对答案。 参考答案要点：

- 第 1 题：dL/dw1 = dL/dy · (1-y²) · w2 · (1-h²) · x，其中 h=tanh(w1x+b1)、y=tanh(·)。维度全部与各自变量同形。

- 第 2 题：softmax_i = e^{z_i}/Σe^{z_j}；合并算子是因为分开实现数值不稳定（上溢）且梯度可化简为 p-y。
- 第 3 题：D_KL(P||Q)=ΣP log(P/Q)=H(P,Q)-H(P)；P 固定时最小化 KL ≡ 最小化交叉熵。
- 第 4 题：**Pre-LN 与 Post-LN 两种都算对**，但必须能说出区别。Post-LN（原论文）：`x = LN(x + Attn(x))` → `x = LN(x + FFN(x))`；Pre-LN（GPT-2 之后主流、训练更稳）：`x = x + Attn(LN(x))` → `x = x + FFN(LN(x))`。形状全程 (B,T,C) 不变，FFN 内部 C→4C→C。写清「LN 在残差加之前还是之后」即可得分。
- 第 5 题：m=β1·m+(1-β1)g；v=β2·v+(1-β2)g²；m̂=m/(1-β1^t)；v̂=v/(1-β2^t)；θ-=lr·m̂/(√v̂+ε)。偏差修正是因为零初始化导致前期估计偏小。
- 第 6 题：scores=QK^T/√d；mask 行加 -inf（softmax 前）；attn=softmax(scores)@V。mask 加在 softmax **之前**。

步骤 2：新建 `day4/week1-gaps.md`，每条错题一行，格式：**「错了什么 → 反映哪个知识空洞 → 补在哪天 → 用什么动作补」**。

错题只分两类，**不许把 A 类拖成 B 类**：

- **A 类 = 本周内能补的**（对应已学/即将学的动手内容）：只有第 1 题（手推梯度）和第 6 题（attention 前向）。→ 分别映射到 **Day 5 任务 1（默写 attention 流水线）** 和 **Day 6 任务 1/3（概念对照 + 对拍）**，动作是「重做一遍原题并写进博客」。
- **B 类 = 本周没教的概念**：第 2/3 题（softmax、CE、KL）、第 4 题（Transformer Block）、第 5 题（Adam）。→ 一律映射到下周的对应主题，本周**不安排补课时间**：

空洞	归到	为什么那天自然会补上
softmax + 交叉熵、`p-y` 梯度	Week 2 · makemore（BigramLM 用 CE/BCE）	损失函数是 makemore 的主角
KL 散度与熵的关系	Week 2 · 采样与分布匹配	与 CE 同一天补，一次讲透
Transformer Block 结构	Week 3 · nanoGPT（手写 Block）	到那天要逐行看 `CausalSelfAttention` + MLP + residual
Adam / 偏差修正 / 学习率	Week 2 · 训练循环与调 lr	那天会现场比较优化器与学习率

硬性要求：至少 6 条，每条都要有「哪天 + 什么动作」。写不出动作的条目视为没诊断出来。

任务 3：知乎第 0 篇成文（约 2h）

1. 打开 `https://zhuanlan.zhihu.com` → 写文章。
2. 标题（三选一或自拟）：「我花三天盲写了一个 micrograd」「别再看视频了:手写反向传播的三个卡点」「从零手写 autograd:梯度为什么会 +=」。
3. 结构照抄（总 1500-2500 字）：① 为什么要手写（2 段）② 盲写卡点 top3（用 NOTES.md 素材，贴关键代码 ≤3 段）③ 数值梯度对拍（贴 gradcheck 输出）④ 决策边界图（贴 moons.png）⑤ 下一步（attention）。
4. 先存草稿，明天任务 5 发布。

完成判据：草稿保存成功，图片 ≥2 张（moons.png + 截图任意）。

任务 4：收工笔记（10 分钟）

NOTES.md：今天自测暴露的最大空洞一句话 + 「它会被哪天补上」。

今日验收：自测 6 题写完 + gaps ≥6 条且每条有日期 + 博客草稿保存。

Day 5 • 周五：手写 attention + 博客发布

今日总览（6h）：今天从 micrograd 跨到 Transformer 的心脏——注意力。纯 NumPy/纯 Python 手写，不依赖 torch。四个任务：单头 → 多头 → 形状与因果性单测 → 热力图可视化。晚上 30 分钟把博客发出去。

任务 1：原理预热（45 分钟）

1. 读一遍下面的极简解释（这就是全部数学，没有更多）：

给定输入矩阵 X（T 个 token，每个 C 维），三个可学习矩阵 W_Q、W_K、W_V 生成查询/键/值。每个 token 拿自己的 Q 去和所有 token 的 K 做点积（相似度），除以 √d 缩放，加 causal mask（第 i 行第 j≥i 列置 -inf，防止偷看未来），softmax 成权重，再对 V 加权求和——输出就是「每个位置对它之前所有信息的加权摘要」。

2. 在 NOTES.md 默写这条流水线（QK^T → scale → mask → softmax → @V），写不出看一眼再写，直到能独立写出。

任务 2：单头 + 多头实现（约 2.5h）

新建 `day5/attention.py`，按 TODO 填空。**先把下面代码里 6 行实现删掉/注释掉，自己填回去再对照**——直接照抄就等于没做（这份文档里的「答案」就是给你对拍用的）。

```
import math
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)      # 数值稳定
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def single_head_attention(X, Wq, Wk, Wv, causal=True):
    """X:(T,C)  Wq/Wk/Wv:(C,d)  返回 (T,d)"""
    Q = X @ Wq                                  # TODO 1: 生成 Q
    K = X @ Wk                                  # TODO 2
    V = X @ Wv                                  # TODO 3
    scores = Q @ K.T / math.sqrt(Q.shape[-1])   # TODO 4: 缩放点积
    if causal:                                  # TODO 5: causal mask
        T = X.shape[0]
        scores += np.triu(np.full((T, T), -np.inf), k=1)
    A = softmax(scores)                         # TODO 6
    return A @ V, A                             # A 留着可视化

def multi_head_attention(X, Ws, Wo, causal=True):
    """Ws: 列表,每个元素是 (Wq,Wk,Wv);Wo:(h*d, C)。各头结果拼接后过 Wo"""
    heads, amaps = [], []
    for Wq, Wk, Wv in Ws:
        out, A = single_head_attention(X, Wq, Wk, Wv, causal)
        heads.append(out); amaps.append(A)
    return np.hstack(heads) @ Wo, amaps
```

完成判据：删掉实现后独立填完 6 个 TODO，`python -c "from attention import single_head_attention"` 不报错。

> 注：这份实现**故意只有 (T,C)、没有 batch 维，也没有位置编码**。要知道这两点差别：真实 Transformer 是 (B,T,C)，多头是靠把 C 切成 h 份再并行做的（这里用「多个独立 (C,d) 权重 + hstack」等价模拟）；没有位置编码的 self-attention 对 token 顺序是置换不变的——causal mask 只挡住了未来，顺序信息在 makemore/nanoGPT 里另外由位置嵌入提供。这一条写进 NOTES，Week 3 会用到。

任务 3：单测（约 1.5h）

新建 `day5/test_attention.py`：**先自己写**这 4 个断言（形状、上三角权重为 0、每行权重和为 1、多头输出形状回到 (T,C)），卡住再看下面的参考实现。

```
from attention import single_head_attention, multi_head_attention
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(5, 8))                    # T=5, C=8

def test_shape():
    out, A = single_head_attention(X, rng.normal(size=(8, 6)), rng.normal(size=(8, 6)), rng.normal(size=
(8, 6)))
    assert out.shape == (5, 6) and A.shape == (5, 5)

def test_causal():
    _, A = single_head_attention(X, rng.normal(size=(8,4)), rng.normal(size=(8,4)), rng.normal(size=
(8,4)))
    assert np.allclose(np.triu(A, k=1), 0)      # 上三角(未来位)权重必须为 0

def test_rows_sum_one():
    _, A = single_head_attention(X, rng.normal(size=(8,4)), rng.normal(size=(8,4)), rng.normal(size=
(8,4)))
    assert np.allclose(A.sum(axis=-1), 1)       # 每行权重和为 1

def test_multi_head():
    Ws = [(rng.normal(size=(8,4)), rng.normal(size=(8,4)), rng.normal(size=(8,4))) for _ in range(3)]
    out, _ = multi_head_attention(X, Ws, rng.normal(size=(12, 8)))
    assert out.shape == (5, 8)

for t in [test_shape, test_causal, test_rows_sum_one, test_multi_head]:
    t(); print(t.__name__, "PASS")
print("ALL TESTS PASS")
```

完成判据：打印 4 个 PASS + `ALL TESTS PASS`。

> 注（强烈建议加第 5 个测试）：`test_causal_output`——把 `X[4]` 换成随机值，重跑后断言前 4 行输出与换之前完全一致。前 4 个测试只查了权重矩阵，这个才真正验证「未来的 token 影响不到过去」，也是 Day 6 与 torch 对拍时最有用的那一条。

任务 4：热力图可视化（约 45 分钟，可砍项）

新建 `day5/visualize.py`：用 matplotlib 把一个头的注意力矩阵 A 画成 5×5 热力图（`plt.imshow(A)`），保存 `day5/attn.png`。

看图要确认的是这三件**必然成立**的事（写进 NOTES.md）：① 上三角全黑（权重 0）② 左上角 `A[0,0]` 必为 1——第一个 token 只能看自己，softmax 归一化后只剩它 ③ 每行的亮格分布是**随机的**。

> 注：不要期待「最亮格在对角线附近」。随机初始化的 Wq/Wk 之下，注意力权重没有局部性可言；「attend 到邻近 token」是训练收敛后才出现的现象，不是 attention 公式的性质。如果你的笔记里写了这条，那是把结论安到了前提上——删掉它，顺便记住这个思维错误（后训练面试里很爱考「你确定这是必然还是巧合」）。

任务 5：博客发布（30 分钟）

打开昨天草稿，补一段「下一步:手写 attention」+ `attn.png`，点发布。**把链接贴到 NOTES.md 顶部**。

今日验收：4 个单测 PASS + 博客已发布（有链接）。attn.png 若已砍则不计。

Day 6 • 周六：PyTorch 桥接 + 对拍 + 周复盘

今日总览（7h）：今天把手写的世界和 PyTorch 的世界接起来——用 `torch.nn` 重写 moons 分类器和 attention，并与自己的手写版对拍（数值一致是硬判据）。晚上做周复盘。这是「我懂原理」到「我会用框架」的最后一公里。

> 注：今天需要 GPU 吗？不需要，「开始前」步骤 0 装的 CPU 版 torch 足够。想顺手体验 device 迁移（`tensor.to('cuda')`、`nvidia-smi` 看显存占用），Day 1 那台实例开 30 分钟跑一遍 torch 版 moons 就行，多花约 ¥1，属可选项。

任务 1：概念对照表（45 分钟）

在 NOTES.md 写一张表（先默写后对照；写不出 optimizer / zero_grad 那两行，说明 Day 2、3 踩过的「梯度没清零」还没真正消化）：

micrograd	PyTorch	一句话关系
Value.data	tensor（requires_grad）	都存数值
Value.grad	tensor.grad	反向后自动填充
`Value._prev` / `_backward` 闭包	`tensor.grad_fn` / 反向图	手写记账 vs torch 建的动态图
backward() 拓扑循环	loss.backward()	torch 内置了拓扑排序
需要手动 `p.grad = 0`	`optimizer.zero_grad()`	不清零就累加，同一个坑
MLP 类	nn.Module + nn.Linear	torch 帮你管参数注册
p.data -= lr*p.grad	torch.optim.SGD.step()	优化器抽象
hinge loss 手写	`nn.BCEWithLogitsLoss` / `F.cross_entropy`	后者内部已做 softmax 数值稳定

任务 2：torch 重写 moons 分类器（约 2h）

新建 `day6/train_moons_torch.py`：用 `nn.Linear` + `tanh` 搭 2-16-16-1 网络，`BCEWithLogitsLoss` 或 hinge、`torch.optim.SGD` 或 Adam，训练到 acc≥95%。要求：

- 与 day2 相同的数据（`make_moons(n_samples=100, noise=0.15, random_state=42)`）。
- 代码量预期 ≤50 行（体会框架帮你省了什么）。

完成判据：acc ≥95%；NOTES.md 写两行「torch 帮我做了什么/我要自己做什么」。

> 注：acc 判定在 torch 里同样别踩 bool 坑——用 `((logits > 0).long() == y).float().mean()`，别写 `(pred > 0) == y`（y 是 ±1 时负类永远判错，上限 50%）。
> 注（15 分钟，**这 15 分钟是本周和后训练最相关的一段**）：把 loss 从 hinge 换成 `F.cross_entropy(logits, y_long)`（y 用 0/1）再跑一遍，记下两件事：① 同样的网络结构，CE 比 hinge 收敛快还是慢 ② CE 的入参是 **logits 而不是概率**（内部做

> 了 log-softmax，这就是自测第 2 题「为什么要合并算子」的工程答案）。SFT / DPO / RLHF 的损失函数全是 CE 家族，第一周不练它一下容易下周陌生。

任务 3：torch attention 与手写版对拍（约 1.5h）

新建 `day6/attention_torch.py`（先把 `day5/attention.py` **复制一份到 day6/**，这样 `from attention import ...` 才成立——用 sys.path 也行，但别在导入上耗时间）：

```
import math
import numpy as np
import torch
from attention import single_head_attention      # day5 那份纯 numpy 实现

def torch_attention(X, Wq, Wk, Wv):
    f64 = lambda t: torch.tensor(t, dtype=torch.float64)   # 必须 float64,否则 1e-10 的容差没意义
    X, Wq, Wk, Wv = f64(X), f64(Wq), f64(Wk), f64(Wv)
    T = X.shape[0]
    mask = torch.triu(torch.full((T, T), float("-inf"), dtype=torch.float64), 1)
    A = torch.softmax(X @ Wq @ (X @ Wk).T / math.sqrt(Wq.shape[1]) + mask, dim=-1)
    return (A @ (X @ Wv)).numpy(), A.numpy()

# 同一组权重喂给两个实现,输出应 allclose
rng = np.random.default_rng(1)
X = rng.normal(size=(6, 8))
Wq, Wk, Wv = (rng.normal(size=(8, 5)) * 0.5 for _ in range(3))
mine, _ = single_head_attention(X, Wq, Wk, Wv, causal=True)
ref, _ = torch_attention(X, Wq, Wk, Wv)
print("max_diff =", np.abs(mine - ref).max())
assert np.allclose(mine, ref, atol=1e-10), "对拍失败"
print("ATTENTION CROSSCHECK OK")
```

完成判据：打印 `ATTENTION CROSSCHECK OK`（浮点误差 atol=1e-10 内）。push 到 GitHub 仓库。

> 注：对拍失败先查三处——① scale 用的是 `d` 的平方根还是 `C` 的（两边必须同一个）② mask 的 k=1（有没有把对角线也挡掉）③ 一边 float32 一边 float64。把 max_diff 打出来比只 assert 有用得多，这是习惯。实测这份对拍的 max_diff 在 1e-16 量级。
>
> 注（Anaconda/Windows 必踩）：`import torch` 报 `OMP: Error #15 ... libiomp5md.dll already initialized`，是 conda 的 MKL 和 torch 自带的 OpenMP 打架。**正解：用「开始前」步骤 0 那个 venv，别在 conda base 里装 torch**；急着往下做就先 `set KMP_DUPLICATE_LIB_OK=TRUE`（Windows cmd）或 `export KMP_DUPLICATE_LIB_OK=TRUE`（Git Bash）绕过，然后继续。

任务 4：周复盘 review.md（约 1h）

新建 `day6/review.md`，写五块：① 本周硬产出清单（贴链接/截图路径）② 自测 6 题现在还有哪题不会 ③ 一周时间真实花销（对照 41.5h，按天记）④ 最有价值的一件事 ⑤ 下周最想补的一个洞。

任务 5：gaps 清账（约 45 分钟）

过一遍 `day4/week1-gaps.md`，每条已补的标 `[x]` 并写一句「怎么补的」；没补的确认都映射到了下周。**A 类洞（第 1、6 题）必须在本周内闭合**，没闭合就占用 Day 7 的前 1 小时。

任务 6：缓冲（约 45 分钟，前面顺就提前收工）

今日验收：torch moons acc≥95% + CROSSCHECK OK + review.md + gaps 全部有状态。

Day 7 • 周日：缓冲补漏 + 总验收 + 下周预习

今日总览（4h）：今天不排新内容。上午按 gaps 清单补漏 + 闭卷复测，中午做总验收，下午 30 分钟预习下周。之后——彻底休息。

任务 1：补漏（约 2h）

打开 week1-gaps.md 和 review.md，按优先级清洞。**规则：最多补 2 个洞，贪多必失**（优先补 A 类：手推梯度、attention 前向）。

任务 1.5：闭卷复测（约 25 分钟，别跳）

补完洞当天就重做一遍：**拿出 Day 4 的原始 6 题，闭卷重做第 1、6 两题**（本周该闭合的那两道），和四天的答卷对比。

> 注：这一步是整个计划里唯一真正检验「补上了没有」的环节。只看 gaps.md 上打了 `[x]` 不算数——「以为懂了」在三天后依然会掉坑，这是学习曲线的基本事实，不是你的问题。复测结果（对/错/半对）写进 review.md 第 ② 块。

任务 2：总验收（30 分钟）

逐项勾选（输出到 review.md 末尾）：

#	验收项	判据
1	GPU 环境	wandb week1-env 有记录（周末补传的 offline 也算）；能独立开机/关机/找到控制台
2	micrograd 跟写	day2 三文件齐全，`train_moons.py` 跑通、acc≥90%（≥95% 记满分）
3	盲写复刻	GitHub 仓库可访问，mine.py 收敛，gradcheck OK
4	知识自测	week1-gaps.md ≥6 条、每条有「哪天+动作」，且第 1/6 题闭卷复测通过
5	公开输出	知乎文章链接可访问
6	手写 attention	4 单测 PASS + CROSSCHECK OK
7	周复盘	review.md 五块齐全

≥6 项通过 = 本周合格；不足 6 项，把缺的写进下周 Day 1 的前 2 小时。

任务 3：下周预习（30 分钟）

打开 Karpathy 的下一个视频「The spelled-out intro to language modeling: building makemore」看前 20 分钟，只做一件事：搞清楚 makemore 和 micrograd 的关系（bigram 语言模型是下一周的起点）。

> 注：顺手在 review.md 末尾写一行「本周在整条路线上的位置」：micrograd（会算梯度）→ makemore（会定义语言建模的损失）→ nanogpt（会搭出 Block）→ **之后才是后训练**：SFT/LoRA、DPO/RLHF、数据配比与评测。第一周不碰任何微调，是有意的：不懂 CE 与 attention 就去调 LoRA，得到的只是不会解释的熟练工。

今日验收：review.md 里验收表 ≥6 项打勾。

三、常见问题与纪律

- **某天没完成怎么办？** 不要熬夜追。把未完成项按「天-任务号」写进 gaps.md，Day 6 的缓冲与 Day 7 就是给它们准备的。连续两天全崩则砍掉 Day 5 的可视化任务（保主线）。**永远优先保 Day 3 盲写和 Day 4 自测**——它们才是这周真正产生能力的两天，画图、发布都是包装。
- **卡住多久该求助？** 单点超过 40 分钟：先写 NOTES 记录，然后搜（Google/官方文档/原仓库 issue），再 20 分钟仍无解就跳过并在 gaps 里立条目。**跳过不丢人，欠账不留名才可怕。**
- **GPU 费用**：只有 Day 1 用卡，每天结束**关机**（不是销毁）；本周总花费应 **≤¥20**，超了就是忘关机，去看 AutoDL 控制台的计费明细。
- **时间不够 6 小时的天**：优先保「硬产出」栏里的东西；视频中可 1.25 倍速，但代码跟写段落必须原速。
- **要不要二刷视频？** 不要整片重看。只有两种情况值得回去：① 盲写时卡住的某个概念（精确跳看那 3-5 分钟）② Day 7 复测第 1 题仍写不出。二刷是「看懂的错觉」最舒服的养法，用做题代替。
- **别提前用 AI 直接生成代码**：micrograd 只有 100 行，让 AI 写一遍等于这周白做。允许 AI 做的事：解释报错、讲清一个概念、review 你写完的文件（**在 Day 3 盲写结束之后**）。